highest ratings returned (the rating is greater than, say, 9). We can use scripting features when adding such ad hoc fields on demand.

To use the scripting feature, append the query with the `script_fields` object at the same level with the required name of the new dynamic file and the logic to populate it. The following listing creates a new field, `top_rated_movie`, by setting a flag based on the ratings the movie receives.

Listing 8.29 Source filtering with a `script` field

```
GET movies/_search
{
  "_source": ["title*","synopsis", "rating"],
  "query": {
    "match": {
      "certificate": "R"
    }
  },
  "script_fields": {
    "top_rated_movie": {
      "script": {
        "lang": "painless",
        "source": "if (doc['rating'].value > 9.0) 'true'; else 'false'"
      }
    }
  }
}
```

The script consists of the `source` element where the logic of populating the new field (`top_rated_movie`) is defined: we stamp the movie as top-rated if its rating is greater than 9. For completeness, let's look at the output (edited for brevity) with the new `top_rated_movie` field:

```
"hits" : [{
  ...
  "_source" : {
    "rating" : "9.3",
   "synopsis" : "Two imprisoned men bond ...",
   "title" : "The Shawshank Redemption"
  },
  "fields" : {
    "top_rated_movie" : ["true"]
  }
}
...
```

SOURCE FILTERING

Earlier, we set the `_source` flag to `false` to suppress documents from being returned in the response. Although we showed all-or-nothing scenarios, there are a couple of use cases where we can implement the `_source` option to tweak the response further. For example, the following listing sets `_source` to `["title*", "synopsis", "rating"]`